

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю № 2

Вариант № 23

Вариант запросов В

Выполнил:
Студент группы ИУ5-34Б
Кострыкина Е. Г.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Задание рубежного контроля

Рубежный контроль представляет собой разработку тестов на языке Python.

- 1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.
- 2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Код программы

main.py

```
from operator import itemgetter

class ProgrammingLanguage:
    """Язык программирования"""
    def __init__(self, pl_id, name):
        self.id = pl_id
        self.name = name

class SyntaxConstruction:
    """Синтаксическая конструкция"""
    def __init__(self, sc_id, name, compile_time, pl_id):
        self.id = sc_id
        self.name = name
        self.compile_time = compile_time
        self.pl_id = pl_id

class LanguageConstruction:
    """Конструкции языков для связи многие-ко-многим"""
    def __init__(self, sc_id, pl_id):
        self.sc_id = sc_id
        self.pl_id = pl_id

# Языки программирования
languages = [
    ProgrammingLanguage(1, "Python"),
    ProgrammingLanguage(2, "Java"),
    ProgrammingLanguage(3, "C++"),
    ProgrammingLanguage(4, "JavaScript")
]

# Синтаксические конструкции
constructions = [
    SyntaxConstruction(1, "Лямбда-функция", 3, 1),
    SyntaxConstruction(2, "Интерфейс", 8, 2),
    SyntaxConstruction(3, "Абстрактный класс", 10, 2),
    SyntaxConstruction(4, "Шаблон функции", 12, 3),
    SyntaxConstruction(5, "Асинхронная функция", 7, 4)
]

# Связи многие-ко-многим
language_constructions = [
    LanguageConstruction(1, 1),
    LanguageConstruction(2, 2),
    LanguageConstruction(3, 2),
    LanguageConstruction(4, 3),
    LanguageConstruction(5, 1),
]

def get_one_to_many(lang, construct):
    """Создаёт соединение один-ко-многим между языками и конструкциями"""
    return [(sc.name, sc.compile_time, pl.name)
            for pl in lang
            for sc in construct
            if sc.pl_id == pl.id]
```

```

def get_many_to_many(lang, construct, lang_construct):
    """Создаёт соединение многие-ко-многим между языками и конструкциями"""
    many_to_many_temp = [(pl.name, lc.pl_id, lc.sc_id)
                          for pl in lang
                          for lc in lang_construct
                          if pl.id == lc.pl_id]

    return [(sc.name, sc.compile_time, pl_name)
            for pl_name, pl_id, sc_id in many_to_many_temp
            for sc in construct if sc.id == sc_id]

def task_b1(one_to_many):
    """Задание B1: Фильтрация конструкций, начинающихся на букву 'А'"""
    # Фильтруем конструкции, начинающиеся с 'А'
    result = list(filter(lambda i: i[0].startswith('А'), one_to_many))
    # Сортируем по названию конструкции (первый элемент кортежа)
    return sorted(result, key=itemgetter(0))

def task_b2(lang, one_to_many):
    """Задание B2: Поиск минимального времени компиляции для каждого языка"""
    result = []
    for pl in lang:
        # Находим все конструкции этого языка
        l_constructions = list(filter(lambda i: i[2] == pl.name,
one_to_many))
        if len(l_constructions) > 0: # Если у языка есть хотя бы одна
конструкция
            l_times = [time for _, time, _ in l_constructions] # Извлекаем
времена компиляции
            l_min_time = min(l_times) # Находим минимальное время
            result.append((pl.name, l_min_time))
        # Сортируем результат по минимальному времени компиляции (второй элемент
кортежа)
    return sorted(result, key=itemgetter(1))

def task_b3(many_to_many):
    """Задание B2: Сортировка всех связанных конструкций и языков"""
    return sorted(many_to_many, key=itemgetter(0))

def main():
    # Получаем соединения
    one_to_many = get_one_to_many(languages, constructions)
    many_to_many = get_many_to_many(languages, constructions,
language_constructions)

    print('Задание B1')
    result_1 = task_b1(one_to_many)
    for construction_name, compile_time, language_name in result_1:
        print(f"Конструкция: {construction_name:<25} Язык: {language_name}")

    print('\nЗадание B2')
    result_2 = task_b2(languages, one_to_many)
    for language_name, min_time in result_2:
        print(f"Язык: {language_name:<15} Мин. время компиляции: {min_time}
мс")

    print('\nЗадание B3')
    result_3 = task_b3(many_to_many)

```

```

        for construction_name, compile_time, language_name in result_3:
            print(f"Конструкция: {construction_name:<25} Язык: {language_name}")

if __name__ == "__main__":
    main()

```

test.py

```

import unittest
from main import *

class TestProgrammingLanguages(unittest.TestCase):
    """Класс для тестирования функций"""
    def setUp(self):
        """Настройка тестовых данных перед каждым тестом"""
        self.languages = languages
        self.constructions = constructions
        self.language_constructions = language_constructions
        # Получаем соединения
        self.one_to_many = get_one_to_many(self.languages,
        self.constructions)
        self.many_to_many = get_many_to_many(self.languages,
        self.constructions, self.language_constructions)

    def test_task_b1(self):
        """Проверка задания B1"""
        result = task_b1(self.one_to_many)
        # Проверяем количество найденных конструкций (должно быть 2)
        self.assertEqual(len(result), 2)

        # Проверяем, что все конструкции начинаются с 'A'
        for item in result:
            self.assertTrue(item[0].startswith('A'))

    def test_task_b2(self):
        """Проверка задания B2"""
        result = task_b2(self.languages, self.one_to_many)
        # Проверяем количество языков (должно быть 4)
        self.assertEqual(len(result), 4)

        # Массив для проверки корректности вычисленных минимальных времён
        expected_results = [
            ("Python", 3),
            ("JavaScript", 7),
            ("Java", 8),
            ("C++", 12)]

        for i, (lang_name, min_time) in enumerate(result):
            expected_lang, expected_time = expected_results[i]
            # Проверяем название языка
            self.assertEqual(lang_name, expected_lang)
            # Проверяем минимальное время компиляции
            self.assertEqual(min_time, expected_time)

        # Проверяем сортировку по возрастанию времени компиляции
        times = [time for _, time in result]
        self.assertEqual(times, sorted(times))

    def test_task_b3(self):
        """Проверка задания B3"""
        result = task_b3(self.many_to_many)

```

```
# Проверяем количество связей (должно быть 5)
self.assertEqual(len(result), 5)

# Проверяем сортировку по названию конструкций
construct_names = [item[0] for item in result]
self.assertEqual(construct_names, sorted(construct_names))

    for i, (construct_name, compile_time, lang_name) in
enumerate(result):
        # Проверяем, что конструкция и язык существуют в исходных данных
        self.assertTrue(any(c.name == construct_name for c in
self.constructions))
        self.assertTrue(any(lang.name == lang_name for lang in
self.languages))

if __name__ == "__main__":
    unittest.main()
```

Результаты выполнения

✓ Tests passed: 1 of 1 test – 3 ms

Launching pytest with arguments test.py::TestProgrammingLanguages::test_task_b1

===== test session starts =====

collecting ... collected 1 item

test.py::TestProgrammingLanguages::test_task_b1 PASSED [100%]

===== 1 passed in 0.03s =====